

Laboratory 4: Logical Reasoning for Planning

Using an LLM to Construct and Test a Simple Planning Agent

Eshan Saikia
2023A7PS1024G

The accompanying `logic.ipynb` contains the executable planner and validation evidence. This report records the planning formulation, manual plan, LLM-assisted implementation, tests, and reflections.

1 Planning Problem Specification

The warehouse has locations A, B, C . The robot and package begin at A , and the goal is to have the package at C :

$$I = \{\text{At}(\text{Robot}, A), \text{At}(\text{Package}, A)\}, \quad G = \{\text{At}(\text{Package}, C)\}.$$

The movement connections are $A \leftrightarrow B$ and $B \leftrightarrow C$.

Action	Preconditions	Effects
Move(X, Y)	At(Robot, X)	$\neg\text{At}(\text{Robot}, X), \text{At}(\text{Robot}, Y)$
PickUp(Package, X)	At(Robot, X), At(Package, X)	$\neg\text{At}(\text{Package}, X),$ Holding(Package)
Drop(Package, X)	At(Robot, X), Holding(Package)	$\neg\text{Holding}(\text{Package}),$ At(Package, X)

Initially, Move(A, B) and PickUp(Package, A) are applicable because their preconditions hold. Drop(Package, C) is not applicable because the robot is not at C and is not holding the package.

A valid manual plan is

$$\text{PickUp}(\text{Package}, A) \rightarrow \text{Move}(A, B) \rightarrow \text{Move}(B, C) \rightarrow \text{Drop}(\text{Package}, C).$$

The corresponding states are:

$$\begin{aligned} S_0 &: \text{At}(\text{Robot}, A), \text{At}(\text{Package}, A) \\ S_1 &: \text{At}(\text{Robot}, A), \text{Holding}(\text{Package}) \\ S_2 &: \text{At}(\text{Robot}, B), \text{Holding}(\text{Package}) \\ S_3 &: \text{At}(\text{Robot}, C), \text{Holding}(\text{Package}) \\ S_4 &: \text{At}(\text{Robot}, C), \text{At}(\text{Package}, C). \end{aligned}$$

2 Agent Design and LLM Use

The state is represented as a set of logical propositions. Each action stores positive and negative preconditions and positive and negative effects. An action is applicable when all positive preconditions are present and all negative preconditions are absent. Applying an action removes its negative effects and adds its positive effects.

Breadth-first search explores the resulting state space. The frontier contains states together with their partial plans, and a visited set prevents repeated exploration of the same state.

The exact implementation prompt supplied to the LLM was:

I want to implement a simple planning agent in Python. Represent a state as a set of logical propositions. Each action should contain a name; positive preconditions; negative preconditions; positive effects; and negative effects. An action is applicable if all positive preconditions are satisfied and all negative preconditions are absent. When an action is applied, remove its negative effects from the state and add its positive effects. Use breadth-first search to find an action sequence that achieves a specified goal. The program should detect when no plan exists, print the resulting action sequence, and print the states reached after each action. Use the three-location warehouse with movement A-B and B-C, the package initially at A, and the goal package-at-C. Keep it dependency-free and easy to inspect.

The generated planner structure was accepted after inspection. I added explicit validation, controlled tests, and state-transition tracing without changing the planning model.

3 Experimental Results

For the original problem, the planner returned the four-action plan shown above. Every action was independently checked for applicability before its transition was accepted, and the final state satisfied $\text{At}(\text{Package}, C)$.

Test	Plan found	Plan length	Valid
Original solvable problem	Yes	4	Yes
No pickup action	No	–	Yes
Robot-only movement	No	–	Yes

In the robot-only test, the robot can reach C , but the package remains at A ; the goal therefore is not satisfied. The impossible test terminates with no plan rather than inventing an unavailable action.

4 Logic and Search

For a state S and action a , the logical component checks

$$S \models \text{Preconditions}(a).$$

If the check succeeds, the action produces

$$S' = \text{Apply}(S, a).$$

The search component then explores the applicable successor states. Thus logic determines which transitions are possible, while BFS determines which legal action sequences are explored.

5 Think About It

An action is not applicable merely because it is listed; every required precondition must hold in the current state. Thus both `Move(A, B)` and `PickUp(Package, A)` are initially applicable, while `Drop(Package, C)` is not.

Designing the state, actions, transitions, goal, and required outputs before prompting an LLM provides a fixed specification against which the generated implementation can be checked. In the program, preconditions are checked by `Action.applicable()`, effects are applied by `Action.apply()`, the goal is tested by `plan_bfs()`, and BFS explores applicable successor states using a FIFO frontier.

The logical component determines which actions are possible in the current state; the search component determines which legal action sequences are explored. The optional LLM self-verification task is not required for the main submission.

6 Reflection Questions

1. **Why specify preconditions and effects before asking an LLM to write the planner?** They define action semantics independently of implementation and provide a precise specification against which generated code can be checked.
2. **What error could occur without precondition checks?** The planner could execute a pickup when the robot and package are in different locations, creating an invalid state transition.
3. **Why is a plan that looks reasonable not necessarily valid?** Every action must be applicable in the state where it occurs, and the final state must satisfy the formal goal.
4. **What did the LLM contribute?** It translated the specified representation and BFS procedure into an executable Python starting point, reducing implementation effort.
5. **What had to be verified independently?** Applicability, effects, goal testing, BFS exploration, failure behaviour, and every action/state transition in the returned plan.
6. **Where is logical reasoning being used?** It is used to determine whether a state's facts satisfy an action's preconditions and to compute the facts in the successor state after applying its effects.
7. **How is planning related to earlier search algorithms?** Planning searches a state space generated by actions, just as other search algorithms explore successor states. Planning adds explicit logical preconditions and effects that define which transitions are available.

7 Optional Extension

The laboratory's Prolog section is explicitly marked optional and is not required for the main planning submission.